DOSSIER DE PASSATION TECHNIQUE

GENTLESEED

Synthèse du projet, état réel, architecture, décisions et feuille de route pour un nouvel agent de développement sur Code Server.

Version de référence	Date du dossier	Plateforme de reprise
v0.7.15 (bêta)	06.11.2026	Code Server / environnement Python

> **Lecture essentielle avant toute modification**
> Le projet fonctionne déjà partiellement en conditions réelles. Les fonctions de comparaison sont présentées en mode **simulation**, mais une **transmission manuelle réelle** vers Radarr ou Sonarr est possible après confirmation explicite. Ne pas confondre ces deux modes et ne jamais déclencher de téléchargement involontaire.

Résumé exécutif

GentleSeed est une interface Web et un moteur d'aide à la sélection des releases multimédias. Il récupère les résultats de recherche de **Radarr/Sonarr** (dont les indexeurs sont généralement pilotés via **Prowlarr**), tient compte des refus/contraintes des applications *arr, classe les releases admissibles selon des préférences (qualité, langue, codec H.264/H.265, seeders), puis permet une sélection manuelle ou, à terme, automatisée. Le téléchargement et l'import restent pris en charge par Radarr/Sonarr et leur client configuré.

Ce que le nouvel agent doit savoir immédiatement

- Dernière archive connue : **GentleSeed_v0.7.15_COMPLET_WIN.zip** (livrée le 05.11.2026). C'est la base à ouvrir dans Code Server.
- Frontend remanié et modulaire, cinq vues ; backend Python encore à refactoriser davantage.
- Dernier incident visible : timeout lors de la recherche **No Game No Life – Saison 00** (8 spéciaux manquants). Ce n'est pas une preuve d'absence de releases.
- Des recherches Radarr et Sonarr et des transmissions manuelles ont déjà réussi ; un test de non-régression est obligatoire.

Sources de cette passation : historique fonctionnel des échanges, captures d'écran du 05.11.2026 et journaux GentleSeed du 05.11.2026. Le contenu source complet de l'archive v0.7.15 n'a pas été inspecté directement dans ce dossier ; les chemins et comportements doivent être confirmés dans le dépôt.

01 / Vision, périmètre et règles métier

Finalité

Éviter le choix automatique de releases mal adaptées, sans réimplémenter les téléchargements ni contourner les exigences de Radarr/Sonarr. GentleSeed doit rendre le raisonnement de sélection **visible, prévisible et contrôlable**.

Flux fonctionnel attendu

- **1.** L'utilisateur sélectionne un média manquant (film, saison ou épisode) dans GentleSeed, ou un mécanisme automatique lui présente un nouveau média.
- **2.** GentleSeed demande les informations et résultats de recherche à **Radarr/Sonarr**. Les indexeurs en amont sont configurés dans l'écosystème *arr, notamment via **Prowlarr**.
- **3.** GentleSeed exploite les métadonnées de chaque release, les motifs de rejet signalés par *arr et les préférences de classement.
- **4.** Il recommande une release admissible ou affiche « aucune release admissible » avec explications.
- **5.** En mode manuel et avec confirmation explicite, l'utilisateur demande l'envoi de cette release à **Radarr ou Sonarr**.
- **6.** Radarr/Sonarr transmettent eux-mêmes au client de téléchargement configuré (par exemple qBittorrent), et gèrent le suivi/import.

> **Règle non négociable**
>
> **Jamais d'envoi direct vers qBittorrent depuis GentleSeed.** L'application reste un sélecteur/orchestrateur : l'autorité de validation, l'envoi au client et l'importation des médias appartiennent à Radarr/Sonarr.

Principes de classement

- Respecter d'abord les contraintes du profil *arr (qualité/cutoff, langue exigée, releases rejetées, minimum de seeders configuré, etc.).
- Classer ensuite les candidats admissibles selon la qualité souhaitée, la langue, les seeders et la **préférence de codec H.264/H.265** (ou automatique).
- La préférence de codec doit être une **pondération**, pas une exclusion stricte de l'autre codec.
- Le choix manuel peut différer de la recommandation ; la raison et le mode de décision doivent apparaître dans les journaux.
- Ne pas implémenter les **packs de films / collections** : décision de périmètre explicite. Les packs **de saison** Sonarr sont un sujet distinct.

02 / Fonctionnalités et expérience utilisateur

Pages et navigation

Vue	Rôle
dashboard	État des intégrations, moteur, résumé de l'activité.
search	Comparaison de releases : application, type de recherche, média manquant, résultats et action manuelle.
logs	Historique, détails de diagnostic et traçabilité des actions.
settings	Connexions, préférences de sélection, localisation et personnalisation.
system	Informations de fonctionnement et commandes de service.

La v0.7.15 introduit une navigation côté navigateur à base de fragment d'URL, par exemple **http://127.0.0.1:8686/#search**. La présence de **#search** est normale : ce n'est pas une erreur de routage serveur.

Identité et préférences

- Direction visuelle **Gentle Mates** : rose/violet, cartes lisibles, logos Radarr/Sonarr, vues plus aérées ; thèmes clair, sombre et automatique, ainsi que variantes de couleur prévues.
- Authentification avec mot de passe choisi par l'utilisateur, **sans mot de passe imposé** ; exigence historique : même un mot de passe d'un caractère est accepté (conserver ce choix, signaler sa faiblesse de sécurité).
- Profil compact sans nom ; menu proposant **Redémarrer**, **Arrêter**, **Se déconnecter**.
- Langues multiples avec choix **automatique ou manuel** ; couverture fonctionnelle et qualité des traductions à contrôler lors de l'audit du code.
- Fuseau horaire détecté ou sélectionné ; format de l'heure **12 h / 24 h**. Convention initiale suisse : **JJ.MM.AAAA HH:mm:ss**, fuseau **Europe/Zurich** (heure d'été/hiver).
- Conserver des **timestamps techniques explicites avec fuseau**, même si l'interface et les exports appliquent un format localisé.

Modes de sélection

Mode	Comportement exigé
Simulation / analyse	Aucun téléchargement lancé ; résultats réels possibles, recommandation simplement calculée.
Manuel réel	Envoi de la release choisie via l'API Radarr/Sonarr après confirmation explicite.
Automatique	Activation volontaire, surveille uniquement les médias concernés ; développement et fiabilisation à vérifier avant déploiement.

> **Attention à la mention « SIMULATION »**
>
> Le badge visible sur la page Recherche ne signifie pas nécessairement que les données sont fictives. Les comparaisons peuvent s'effectuer sur des résultats réels, sans envoi automatique ; l'option d'envoi manuel réel doit rester séparée et explicite.

03 / Architecture technique et déploiement

Pile technique et exécution

Backend **Python** avec **FastAPI/Uvicorn** ; frontend **HTML/CSS/JavaScript** servi localement. Dans le scénario Windows testé, l'application est accessible sur **127.0.0.1:8686**. La v0.7.15 comporte un frontend modulaire avec un script de construction **build_frontend.py**. La présence d'une configuration Docker/TrueNAS est établie, mais son bon fonctionnement sur la version finale doit encore être confirmé.

Arborescence relevée sur la capture Windows v0.7.15

```
GentleSeed/
  data/                # configuration / événements / journaux
  scripts/             # scripts utilitaires / build frontend
  tests/               # tests Python
  web/                 # pages et ressources frontend
  GentleSeed.py        # point d'entrée / logique historique
  WebUI.py             # exposition de l'interface / API
  Settings.py          # configuration / préférences
  SearchDiagnostics.py # diagnostics de recherche
  DeliveryTracker.py   # suivi des transmissions / imports
  Auth.py              # authentification
  InstanceLock.py      # instance unique
  Assets.py            # assets
  Dockerfile; docker-compose.yml
  docker-compose.truenas.yml
  requirements.txt; .env.example
  WIN-Lancer_Windows.bat; WIN-INSTALLATION.md
  README.md; README_DOCKER.md; CHANGELOG.md
  RELEASE_NOTES_0.7.1.md ... RELEASE_NOTES_0.7.15.md
```

Des fichiers complémentaires **WIN-LocalLauncher.py** et **WIN-README_Windows.md** ont aussi été retenus dans les échanges. L'agent doit vérifier leur présence exacte dans l'archive remise.

Stockage et configuration

- Choix actuel : **JSON** pour la configuration et les états, **events.jsonl** pour les événements, fichiers **.log** pour les journaux techniques.
- **SQLite non retenu** à ce stade : éviter toute migration de persistance non demandée.
- Ne jamais publier de clés API, mots de passe, secrets .env ni logs contenant des identifiants ; garder des exemples anonymisés.

04 / Historique et état de livraison

Repères historiques vérifiables ou issus des décisions

Repère	Évolution / état connu
v0.5.0	Ancienne étape du prototype FastAPI, interface Web et configuration des intégrations.
v0.7.9	Travail sur le parcours de recherche par saison/épisode et les démonstrations visuelles (PC/mobile).
v0.7.11	Journaux réels Sonarr : recherches, choix manuel d'un pack saison American Dad, confirmation par l'historique Sonarr.
v0.7.12	Journaux réels Radarr : recherches, choix manuel, suivi, confirmation et import constaté.
v0.7.13–14	Travail sur diagnostics, préférences de langue/date/heure, illustrations et modularisation progressive. À contrôler dans les notes de versions.
v0.7.15	Archive complète Windows, navigation à cinq vues, frontend réorganisé, build, tests annoncés et diagnostics de timeout.

Matrice de maturité - ne pas confondre « présent » et « validé »

Composant	État de passation
Interface 5 vues	**Présente** dans la v0.7.15 ; navigation #search observée.
Radarr / recherches	**Succès constatés** en v0.7.12 ; à revalider en v0.7.15.
Radarr / envoi manuel	**Succès constaté** avec confirmation d'import dans les logs.
Sonarr / recherche et envoi	**Succès constatés** sur une saison ; cas S00 encore en échec/timeout.
Diagnostics de rejet	**Présents** dans les journaux, précision de l'UI à vérifier.
Internationalisation	**Introduite / prévue** ; valider tout le parcours et les exports.
Docker / TrueNAS	**Fichiers présents** ; déploiement final non confirmé.
Automatisation réelle	**Ne pas déclarer validée** sans batterie de tests de sécurité.
Backend modulaire	**Refactorisation restante** ; éviter les régressions.

> **Résultat des tests automatisés (historique de livraison)**
>
> Pour la v0.7.15, la livraison précédente annonçait **160 tests Python + 10 sous-tests**. Ce chiffre ne constitue pas une validation indépendante dans le présent document : l'agent doit exécuter lui-même la suite et consigner ses résultats.

05 / Résultats réels issus des journaux (05.11.2026)

Les exemples ci-dessous proviennent de journaux d'exploitation transmis lors des échanges. Ils démontrent que le circuit peut fonctionner sur des médias standards, sans garantir le comportement de tous les cas limites dans la v0.7.15.

Cas / version observée	Faits établis
Sonarr, American Dad S01 v0.7.11	103 releases retournées, 6 admissibles. Recommandation simulée en H.264 ; **choix manuel d'un pack H.265** transmis, accepté et confirmé dans l'historique Sonarr.
Sonarr, Invincible S04 v0.7.11	68 releases, 1 admissible, sélection simulée produite (H.264).
Radarr, Le Projet Blair Witch v0.7.12	48 releases, 9 admissibles. Sélection manuelle H.265 / 161 seeders acceptée ; release confirmée ; **import confirmé** ultérieurement.
Radarr, Minions and Monsters v0.7.12	80 releases, 26 admissibles. Choix manuel H.265 / 545 seeders, envoi accepté et release confirmée.
Radarr, autre média v0.7.12	137 releases, **0 admissible**. Les refus détaillent cutoff de qualité, langue requise et seeders insuffisants.

Enseignements

- La sélection recommandée et le choix manuel doivent rester indépendants : American Dad en fournit un exemple concret.
- Un résultat avec **0 admissible** est possible même si de nombreuses releases brutes sont trouvées ; l'UI doit afficher les motifs de rejet.
- Un résultat « transmis » n'est pas forcément « importé » : garder des états distincts **accepté par l'API**, **présent en file/historique**, **import confirmé**.
- Les journaux techniques historiques utilisent des timestamps avec offset **+00:00** ; respecter le fuseau au moment de l'affichage local.

Références à conserver avec la reprise

gentleseed-2026-11-05(2).log (cas Radarr et Sonarr), captures de l'écran Recherches en v0.7.15 et archive **GentleSeed_v0.7.15_COMPLET_WIN.zip**. Éviter d'écraser les logs de preuve ; archiver une copie anonymisée avant des essais réels.

06 / Incident ouvert : No Game No Life – Saison 00

Symptôme reproduit sur l'interface v0.7.15

Parcours : **Recherches → Sonarr – Épisodes / Saisons → Saison complète → No Game No Life – Saison 00 (8 épisodes manquants)**. Après lancement, l'interface affiche « **Recherche non terminée** » avec le message indiquant que **Sonarr n'a pas répondu dans le délai prévu**, puis propose « Réessayer ». Un délai API de l'ordre de **60 secondes** a été observé dans la discussion précédente.

> **Ce que ce message signifie / ne signifie pas**
>
> Le timeout n'établit **ni** qu'il existe des releases **ni** qu'il n'en existe aucune. Une saison 00 regroupe des épisodes spéciaux, qui peuvent être nommés autrement ou distribués individuellement : une recherche par saison peut solliciter plusieurs recherches coûteuses.

Hypothèses à distinguer

- Délai de réponse côté Sonarr/API, indexeur lent, requêtes successives ou attente excessive dans GentleSeed.
- Saison 00 moins bien couverte par les indexeurs ; titres/noms de releases atypiques ou épisode spécial introuvable.
- La recherche globale peut ne pas suivre le même chemin API qu'une recherche unitaire, ou gérer imparfaitement le timeout.
- Le libellé **SIMULATION** de l'UI peut créer une ambiguïté sur ce qui est réellement interrogé ; le clarifier si nécessaire.

Diagnostic minimal sans déclencher de téléchargement

- Tester d'abord une **saison ordinaire** (S01+) avec épisodes manquants, puis un **épisode isolé**. Noter la durée, le nombre de résultats et le détail des refus.
- Reproduire la recherche de **S00** en contrôlant les journaux Sonarr et GentleSeed ; examiner le temps pris par chaque étape et la réponse API exacte.
- Vérifier un **S00E01** individuellement, puis un autre spécial. Comparer avec la recherche de saison et l'activité des indexeurs.
- Distinguer dans la réponse backend **« 0 release trouvée »**, **« toutes refusées »**, **« erreur réseau/API »** et **« timeout »**.
- Rendre le délai d'attente configurable et tracer les durées sans augmenter arbitrairement les timeouts pour masquer le blocage.

Critère de résolution : aucune recherche interrompue ne doit être présentée comme « aucun résultat ». Un timeout doit produire une erreur explicite et une trace exploitable, et les autres recherches doivent continuer à fonctionner.

07 / Batterie de tests de non-régression

Exécuter les tests ci-dessous sur la **v0.7.15 telle quelle**, avant toute refactorisation. Commencer en lecture seule / simulation. N'autoriser un envoi réel qu'après un choix explicite sur un média de test.

ID	Scénario	Attendu
T01	Démarrage Windows / connexion / cinq vues	Aucune erreur JS ou backend, URL en #route utilisable.
T02	Connexions Radarr/Sonarr/Prowlarr	Statuts cohérents, aucun secret exposé dans les logs.
T03	Radarr : film avec releases admissibles	Classement lisible, détails codec/seeders et motifs.
T04	Radarr : 0 release admissible	Compte des releases refusées et explications.
T05	Sonarr : saison S01+	Résultats ou refus, sans timeout anormal.
T06	Sonarr : épisode individuel	Résultat cohérent avec Sonarr, durée relevée.
T07	Sonarr : saison 00 (8 spéciaux)	Réponse classifiée même si la recherche échoue.
T08	Classement : codec, qualité, langue, seeders	Préférence codec non bloquante ; rejet *arr respecté.
T09	Choix manuel vs recommandation	Aucune confusion, confirmation avant vrai envoi.
T10	Envoi test + suivi côté *arr	API acceptée, suivi file/historique/import distinct.
T11	Logs, dates, langues, heure 12/24 h	Europe/Zurich correct ; timestamps UTC conservés.
T12	Build frontend / tests Python / Docker	Artefacts générés ; tests documentés ; Docker à valider.

Traces à capturer pour chaque test

- Version, système, mode de sélection, média concerné (sans exposer d'information privée).
- Horodatage local + timestamp technique avec offset, durée totale, endpoints ou étape échouée si connu.
- Releases trouvées / admissibles / rejetées et motifs ; capture d'écran ou extrait de journal anonymisé.
- Pour les envois réels : preuve de confirmation utilisateur et réponse de Radarr/Sonarr ; jamais tester sur un média non convenu.

08 / Feuille de route pour la suite

P0 — Stabilisation et preuve de fonctionnement

- Exécuter intégralement la batterie de tests en v0.7.15 et documenter les écarts, notamment No Game No Life S00.
- Instrumenter les phases des recherches Sonarr : début/fin, volume de requêtes, durée des appels, réponse des indexeurs si accessible et timeout de bout en bout.
- Fiabiliser la classification de résultats : trouvés, admissibles, tous refusés, erreur, timeout ; ne jamais lancer d'envoi en mode simulation.
- Maintenir intact le parcours de transmission manuelle vers Radarr/Sonarr avec confirmation, suivi et import.

P1 — Architecture, maintenance et cohérence fonctionnelle

- Refactoriser progressivement le backend Python monolithique en services ou modules testables, sans changer silencieusement les contrats du frontend.
- Consolider les tests unitaires, API et intégration : comparaison, parsing des releases, limites Sonarr, suivi de delivery, permissions/auth.
- Vérifier le build frontend, la navigation, le fonctionnement des boutons, les messages d'erreur, les états vides et le comportement sur mobile.
- Auditer toutes les préférences de langue, fuseau et format 12/24 h, y compris journaux et exports.

P2 — Déploiement et confort

- Valider les scénarios de déploiement Windows et Docker/TrueNAS, les chemins persistants, la mise à jour et le redémarrage propre.
- Améliorer les temps d'attente configurables, les indicateurs d'avancement et les diagnostics de performance.
- Envisager, seulement après stabilisation, l'automatisation contrôlée pour nouveaux médias, avec paramètres explicites, limites et garde-fous.

> **Version future**
>
> La prochaine itération peut être nommée **v0.7.16** si elle contient les corrections suivantes, mais **aucune release v0.7.16 n'est considérée comme livrée** par ce dossier.

09 / Consignes de reprise dans Code Server

Documents et artefacts à fournir à l'agent

- Archive de travail : **GentleSeed_v0.7.15_COMPLET_WIN.zip**.
- Captures UI de la version actuelle, et journaux de test du 05.11.2026 si nécessaire, **après masquage des secrets**.
- Consulter **README.md**, **CHANGELOG.md**, **RELEASE_NOTES_0.7.15.md**, **WIN-INSTALLATION.md** et les fichiers de tests présents dans l'archive.

Méthode de travail demandée

- **Auditer avant de modifier :** cartographier les modules Python, routes API, modules JS, fichiers de config, tests et dépendances.
- **Établir un état de référence reproductible :** installer les dépendances dans l'environnement ciblé, démarrer l'application, exécuter les tests actuels.
- **Proposer de petites corrections isolées :** d'abord observabilité et timeout Sonarr, puis refactorisation avec tests de non-régression.
- **Livrer pour chaque évolution :** diff de fichiers, protocole de test, résultats avant/après, notes de version et procédure de mise à jour.
- **Préserver les données :** sauvegarde de data/ avant migration ; ne jamais écraser les paramètres, tokens ou historique lors des mises à jour.
- **Ne pas lancer de téléchargements réels sans validation explicite** de la personne qui utilise GentleSeed.

Instructions prêtes à transmettre au nouvel agent

> **BRIEF AGENT — À COPIER**
>
> Tu reprends le projet GentleSeed à partir de l'archive v0.7.15. Commence par lire le dossier de passation, puis audite le code réel et corrige les éventuels écarts entre documentation et implémentation. N'entreprends aucune refonte totale sans justification : conserve la structure frontend modulaire, le comportement Windows, les données JSON/events.jsonl/logs et la compatibilité Radarr/Sonarr. Règles impératives : GentleSeed ne télécharge jamais directement via qBittorrent ; la validation et l'envoi passent toujours par Radarr/Sonarr ; simulation par défaut, envoi manuel réel uniquement après confirmation explicite ; respect des refus *arr et préférence codec H.264/H.265 non éliminatoire. Ne développe pas les packs de films/collections. Priorité : batterie de tests v0.7.15, puis diagnostic du timeout Sonarr sur No Game No Life S00, ensuite refactorisation progressive du backend avec non-régression. Vérifie la localisation, Europe/Zurich, les formats 12/24 h et les timestamps techniques explicites. Avant chaque correction, présente les hypothèses et tests qui permettront de la valider, puis fournis un changelog et les fichiers modifiés.

Point de départ recommandé : ouvrir **GentleSeed_v0.7.15_COMPLET_WIN.zip** dans Code Server, exécuter la suite de tests, puis reproduire un test saison standard Sonarr et le test S00 sans déclencher de téléchargement.